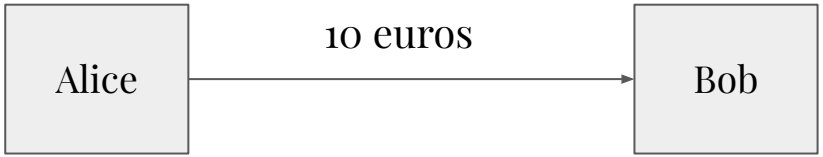
Just one proof away

The cryptography behind private payments

Giorgio Dell'Immagine · Cryptography engineer at zkSecurity

Payments



Options:

- **Banking system**, centralized, not private
- **Blockchains** (for example Bitcoin): decentralized, but everything is public

Can we do better?

Private payments

Our goal is to build a system that

- Is **decentralized** and **non-censorable**
- **Hides every detail about a transaction**: the sender, the receiver and the amount

This would be the most similar thing we currently have to digital cash.

It turns out that this is possible, using some really interesting **cryptography**!

What we'll cover in this talk

1. Introduce some cryptographic primitives
2. Build the minimal protocol that implements private payments
3. What if there are bugs?

Commitment schemes

Cryptographic tools that support two operations

- `commit(value, blinding)` returns a commitment to a value using blinding as additional randomness
- `verify(commitment, value, opening_proof)` verifies a commitment opening proof against the original commitment and .

A commitment scheme must satisfy two properties:

- **Hiding**: the commitment does not reveal anything about the underlying value.
- **Binding**: it is infeasible to produce an opening proof for a value different than the one it was originally committed to.

Hash-based commitment scheme

The simplest commitment scheme is a hash-based one.

```
def commit(value, blinding):
    return H(value, blinding)

def verify(commitment, value, blinding):
    return commitment == H(value, blinding)
```

Binding and hiding follows easily from the security of the cryptographic hash function.

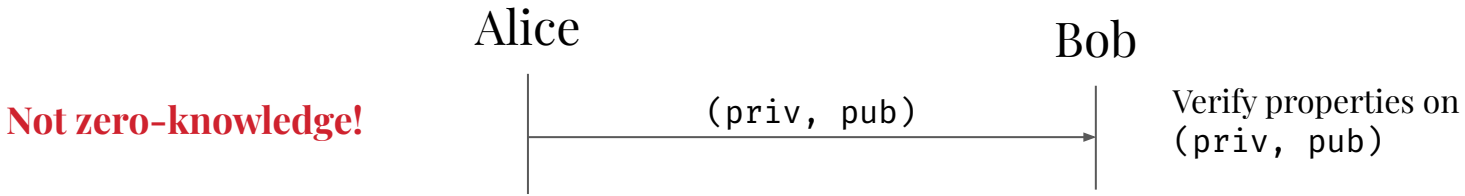
Zero-knowledge proofs

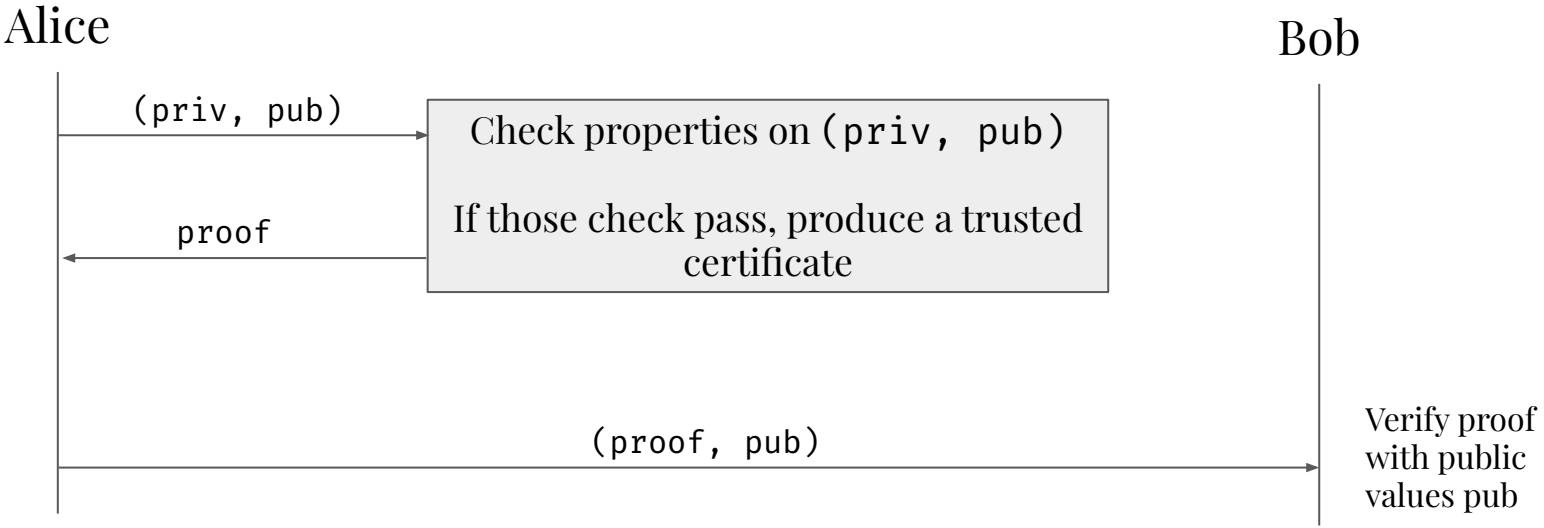
Zero-knowledge proofs are a powerful cryptographic primitive that allows an **untrusted prover** to produce a **certificate of knowledge** of some values that satisfy some fixed and public properties. A verifier can then check that the proof is correct.

We require two properties:

- **Knowledge soundness**: if the verifier accepts, then the prover really knew some values that satisfy the given properties.
- **Zero-knowledge**: the proof does not reveal anything about the values provided by the prover, except that they satisfy the properties.

Zero-knowledge proofs





What statements we will prove

We will be interested in proving statements like this:

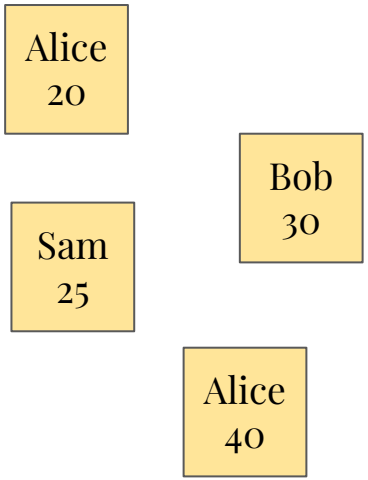
```
I know private values (priv_1, priv_2, …, priv_n) such that
for public values (pub_1, pub_2, …, pub_m) it holds that
    1. Property 1
    2. Property 2
    …
    K: Property k
```

Note-based ledgers (aka UTXO)

We store money into **notes**, which is just a tuple `(address, value)`. Notes represent an amount that can be spent by the owner of the private key for the stored address.

The **balance** of some address is the sum of the values of all unspent notes owned by that address.

We assume that we have a consensus layer that keeps track of the set of all notes and all the spent notes.



Transactions in note-based ledgers

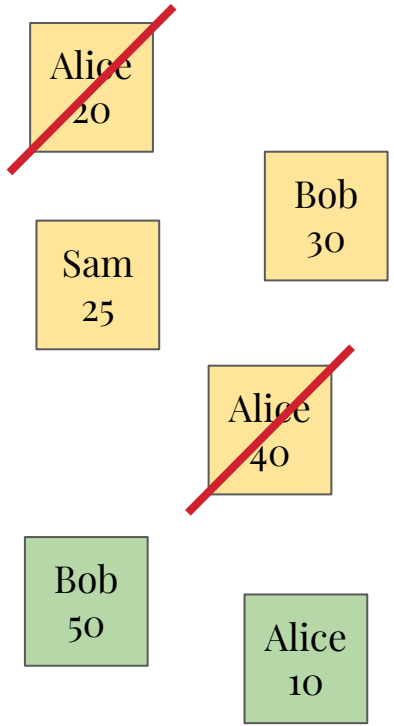
A transaction is the act of

- Spending some input notes.
- Creating some output notes.

Suppose Alice wants to send 50 to Bob

The transaction:

- Spends the note `(Alice, 20)`
- Spends the note `(Alice, 40)`
- Creates a new note `(Bob, 50)`
- Creates a new note `(Alice, 10)`



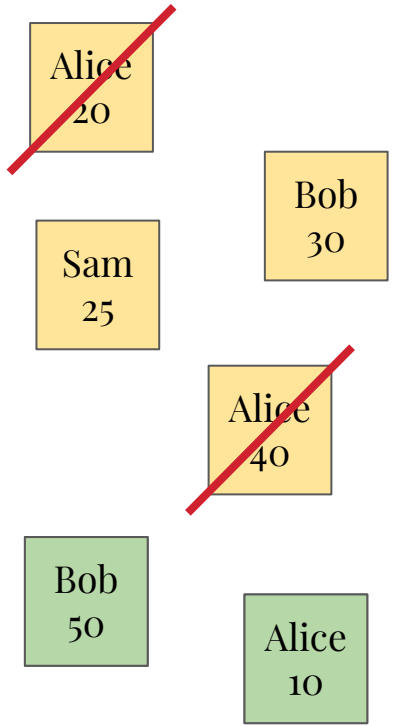
Verifying transactions in note-based ledgers

```
def verify_tx(input_notes, signatures, output_notes):
    # check that the input notes are valid, owned and unspent
    for note in input_notes:
        assert note is in the notes set
        assert note is not marked as spent
        assert signature[i] for note is valid

        mark note as spent

    # balance integrity
    input_value = sum of values of input_notes
    output_value = sum of values of output_notes
    assert input_value == output_value

    # insert outputs
    insert output_notes in the notes set
```



Adding privacy

- **Notes must be encrypted**.
- We need to **hide the sender**: we should not be able to recover which notes were spent in a transaction.
- We need to **hide the receiver**: output notes should be encrypted.
- We need to **hide the amounts** transferred.

First idea: note commitments

Instead of storing a note as a clear-text pair (address, value), let's store commitments to them

```
note_commitment = H(address, value, blinding)
```

- The **hiding** property ensures that the commitment does not reveal anything about the underlying note.
- Moreover the **binding** property ensures that the commitment can only be opened to the original note (i.e., the note cannot be modified).

Ok, but how do we do transactions?

Reminder: verifying transactions in note-based ledgers

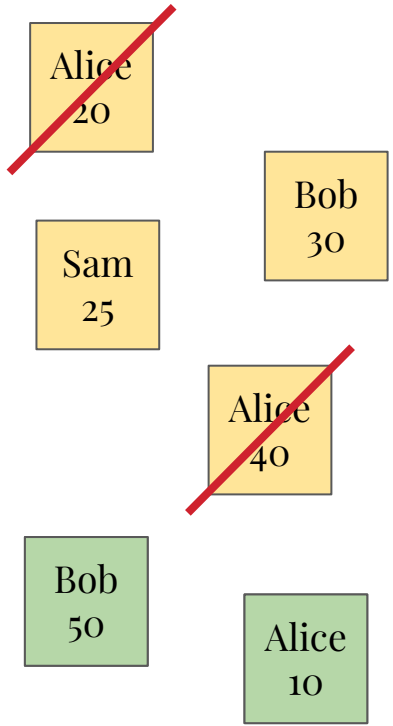
```
def verify_tx(input_notes, signatures, output_notes):
    # check that the input notes are valid, owned and unspent
    for note in input_notes:
        assert note is in the notes set
        assert note is not marked as spent
        assert signature[i] for note is valid

        mark note as spent

    # balance integrity
    input_value = sum of values of input_notes
    output_value = sum of values of output_notes
    assert input_value == output_value

    # insert outputs
    insert output_notes in the notes set
```

We need to do these checks, while hiding everything!

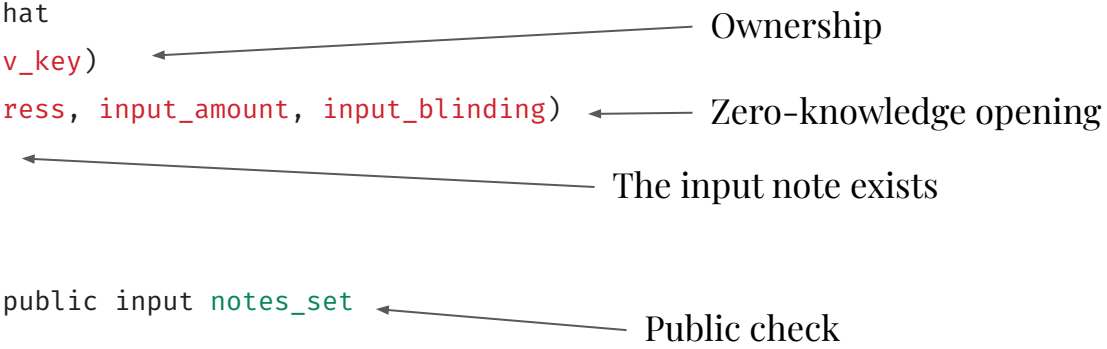


Zero-knowledge proofs!

Let's prove that the input note commitment is in the set of note commitments, and that we know the private key associated with the input address.

```
I know private values (input_address, input_amount, input_blinding, priv_key) such that
for public values (notes_set) it holds that
   1. input_address = derive_address(priv_key)
   2. let input_commitment = H(input_address, input_amount, input_blinding)
   3. input_commitment is in notes_set

verify_tx(proof):
   1. check that the proof is valid for public input notes_set
```



Emitting a new note

We can additionally emit a new note commitment as a public value. The balance integrity is enforced by the proof

```
I know private values (input_address, input_amount, input_blinding, priv_key, ouptut_address, output_value,
output_blinding)  such that
for public values (notes_set, output_commitment) it holds that
   1. input_address = derive_address(priv_key)
   2. let input_commitment = H(input_amount, input_address, input_blinding)
   3. input_commitment is in notes_set
   4. input_value is equal to output_value
   5. output_commitment is equal to H(output_address, output_amount, output_blinding)

verify_tx(proof, output_commitment):
   1. check that the proof is valid for public inputs (notes_set, output_commitment)
   2. insert output_commitment in notes_set
```

Balance integrity

Output commitment

Output insertion

Is this enough?



Double spends, infinite spends

We don't yet have any mechanism to prove that the input note was not already spent! **How to do it without revealing the input notes?**

When we spend a note, we are required to reveal its **unique nullifier**:

- a pseudo-random value which is **deterministically generated** from the commitment,
- but **cannot be publicly linked to the commitment**.

Each nullifier is stored publicly. If one tries to double spend the same note, it will be forced to generate and reveal the same nullifier, so the transaction will be rejected.

```
nullifier = H(commitment, input_blinding)
```

The full protocol

I know private values (input_address, input_amount, input_blinding, priv_key, ouptut_address, output_value, output_blinding) such that

for public values (notes_set, input_nullifier, output_commitment), input it holds that

1. input_address = derive_address(priv_key)
2. let input_commitment = H(input_amount, input_address, input_blinding)
3. input_commitment is in notes_set
4. input_value is equal to output_value
5. output_commitment is equal to H(output_address, output_amount, output_blinding)
6. input_nullfier is equal to H(input_commitment, input_blinding)

Nullifier integrity

verify_tx(proof, output_commitment, input_nullifier):

1. check that the proof is valid for public inputs (notes_set, output_commitment, input_nullifier)
2. check that input_nullifier is not in the nullifier set
3. insert output_commitment in notes_set

Double spend prevention

ZCash and Monero

Zcash and Monero are the two most-known protocols that implement these ideas.

Zcash is very similar to the protocol we described

Monero is a bit different, but the general ideas are the same:

- Uses Ring-CT, which proves that each input is one among 16 notes (moving to FCMP++) and reveals a nullfier
- Uses homomorphic commitments + Bulletproofs to check balance integrity

What if there are bugs?

If there is a bug in Bitcoin, we can immediately detect it, because everything is public.

However, using these protocols, **what if the properties that we check are wrong?** For example we miss out one critical check?

The same zero-knowledge property that hides everything, **hides also if the bug was exploited and by who**.

Recent example: the Orchard counterfeiting vulnerability in ZCash.

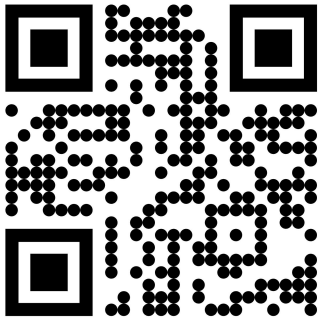
Is high assurance possible?

The set of properties, at the end of the day, is just software. And software has bugs.

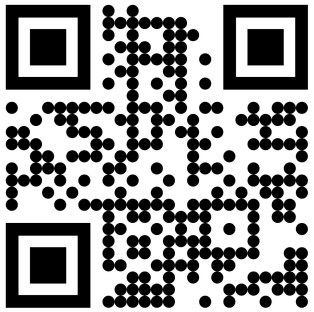
Can we do something better? At **zkSecurity** we help teams gain confidence on critical cryptography

- We do specialized cryptography audits
- Formal verification: Clean
- We recently helped ZCash formally verify that the new checks are correct and do not lead to undetectable bugs (https://github.com/zcash/ironwood)

Thanks for listening!



daltron.de



zksecurity.xyz